SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY
COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO2: Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

Debugging & Optimisation Task : 10%

QUESTIONS: Total Marks: 50 Annexure A

Code of Conduct:

I __BANAGANAPALLI MUNNA(192372028)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:B.Munna

Annexure A
Debugging & Optimisation Task

1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)
A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
B. Optimize the code to reduce execution time for large input datasets (≥10 MB).
C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

2. Performance Analysis of Block Cipher Modes of Operation A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from

high latency and memory inefficiency.

A. Debug incorrect padding and IV handling causing decryption failures. B. Refactor the code to minimize redundant computations and memory usage. C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

3. Debugging RSA Cryptosystem Implementation A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

A. Identify issues in prime generation, key generation, and modular exponentiation.
B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
C. Evaluate the trade-offs between key size, security, and computational performance.

4. Optimization and Security Analysis of Diffie-Hellman Key Exchange A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

A. Debug incorrect shared key generation due to improper modular arithmetic.
B. Optimize exponentiation operations for large prime values.
C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

A. Identify and fix errors in point addition and scalar multiplication logic. B. Optimize the implementation using efficient algorithms (e.g., double-and-add).
C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

Debugging & Optimization of Symmetric Encryption (DES / 3DES)

Scenario

A Python implementation of **DES/3DES** produces different ciphertext for the same plaintext and key across multiple executions. The goal is to identify the errors, optimize performance for large datasets (≥10 MB), and determine whether a modern cipher should replace DES/3DES.

A. Identify and fix logical and implementation errors

Problem Identification

The inconsistent ciphertext indicates implementation errors rather than problems with the DES algorithm itself.

Possible Errors and Fixes

Error	Cause	Fix
Incorrect key scheduling	Wrong generation or ordering of 16 round keys	Generate round keys according to the DES key schedule and use the same order for encryption.
Incorrect permutation tables	Initial Permutation (IP) or Final Permutation (FP) implemented incorrectly	Verify all permutation tables against the DES specification.
Wrong left/right swap	Incorrect swapping after each Feistel round	Swap the left and right halves exactly as defined in DES.
Random Initialization Vector (IV)	New IV generated every run without storing it	Use the same IV during testing or transmit the IV with the ciphertext.
Padding errors	Different padding applied each run	Use a standard padding method such as PKCS#5 or PKCS#7 consistently.

Debugging Result

After correcting the key schedule, permutations, Feistel rounds, IV handling, and padding, the program produces **identical ciphertext for the same plaintext, key, and IV**.

B. Optimize the implementation

Optimization Techniques

1. **Precompute permutation tables**
    - Avoid recalculating permutations for every block.
2. **Use efficient bitwise operations**
    - Replace string-based bit manipulation with integer bitwise operations.
3. **Process data in blocks**
    - Encrypt large files block-by-block instead of loading the entire file into memory.
4. **Reuse round keys**
    - Generate the 16 round keys once and reuse them for all blocks.
5. **Parallel processing**
    - Encrypt independent blocks in parallel (e.g., CTR mode).

Result

- Reduced execution time for datasets larger than **10 MB**.
- Lower CPU usage.
- Improved scalability.

C. Should DES/3DES be replaced?

Comparison

Feature	DES	3DES	AES
Key size	56 bits	112/168 bits	128/192/256 bits
Security	Weak	Moderate	Strong
Speed	Moderate	Slow	Fast
Current recommendation	Not recommended	Being phased out	Recommended

Analysis

- **DES** is insecure because its 56-bit key can be broken by brute-force attacks.
- **3DES** improves security but is significantly slower due to three DES operations.
- **AES** provides stronger security with much better performance and is the current industry standard.

Decision

Replacing DES/3DES with **AES** improves both **security** and **performance**, making it the preferred choice for modern applications.

---

5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation

Scenario

A Python ECC implementation for key exchange produces incorrect results for certain elliptic curve parameters.

A. Identify and fix errors

Problem Identification

The errors usually occur in **point addition** and **scalar multiplication**.

Common Errors and Fixes

Error	Cause	Fix
Incorrect point addition	Wrong slope calculation	Use the correct elliptic curve formulas.
Division errors	Ordinary division instead of modular inverse	Compute division using the modular inverse in finite fields.
Missing point-at-infinity case	Identity element not handled	Return the point at infinity when required.
Point doubling errors	Same formula used for all cases	Use the dedicated point-doubling formula when $P=QP$ = $QP=Q$.
Scalar multiplication errors	Incorrect repeated addition	Implement the Double-and-Add algorithm correctly.

Debugging Result

After correcting point addition, modular arithmetic, point doubling, and identity handling, the ECC implementation produces correct shared keys for all valid curve parameters.

B. Optimize using Double-and-Add

Double-and-Add Algorithm

Instead of repeated addition:

kP=P+P+P+···kP=P+P+P+\cdotskP=P+P+P+···

use:

- Point doubling
- Point addition only when needed

Complexity

- Naive method: **O(k)**
- Double-and-Add: **O(log k)**

Benefits

- Much faster execution.
- Lower CPU usage.
- Better performance for large key sizes.
- Suitable for embedded and IoT devices.

C. Compare optimized ECC with RSA

Feature	RSA	Optimized ECC
Equivalent security	1024-bit key	160-bit key
Key size	Large	Small
Memory usage	High	Low
CPU usage	High	Low
Power consumption	High	Low

Key generation	Slower	Faster
Suitability for IoT	Moderate	Excellent

Analysis

- ECC provides the same level of security as RSA using much smaller keys.
- Smaller keys reduce memory, computation, and communication overhead.
- Optimized ECC with Double-and-Add performs efficiently on low-power processors.

Decision

For **resource-constrained environments** (IoT, embedded systems, smart devices), **optimized ECC** is more suitable than RSA because it offers:

- Equivalent security.
- Faster execution.
- Lower memory usage.
- Reduced power consumption.
- Better overall efficiency.

Final Conclusion

1. DES/3DES Debugging & Optimization

- Fixed errors in **key scheduling, permutations, IV handling, and padding** to ensure consistent ciphertext.
- Optimized performance using **precomputed tables, bitwise operations, block processing, key reuse, and parallelization**.
- Replaced **DES/3DES with AES** for significantly better security and performance.

2. ECC Debugging & Enhancement

- Corrected **point addition, point doubling, modular inverse calculations, and scalar multiplication**.
- Optimized scalar multiplication using the **Double-and-Add algorithm**, reducing complexity from **O(k)** to **O(log k)**.
- Compared with RSA and concluded that **ECC is the preferred cryptosystem** for resource-constrained environments due to its smaller keys, lower computational cost, lower memory usage, and stronger practical efficiency.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation